

Treehouses

Comprimento mínimo de cabo para conectar casas na floresta tropical

Kattis · open.kattis.com/problems/treehouses

Lucas Barreira · José Eugênio · Carlos Huan · Mateus Rocha

Contexto e Objetivo

Existe uma floresta tropical com **N casas** nas copas das árvores. As primeiras **K casas** estão próximas ao solo e já podem se comunicar entre si por terra — sem custo algum.

Além disso, já existem **P cabos aéreos** instalados entre pares de casas. Como cabos cruzados não podem ser trocados, a estrutura existente é permanente.

O objetivo é instalar o menor comprimento possível de cabo novo para que **todas as casas fiquem conectadas** — seja por caminhada, cabo existente ou cabo novo.

N

Casas na árvore

$$1 \leq N \leq 1.000$$

K

Casas com acesso terrestre

$$1 \leq K \leq N$$

P

Cabos já instalados

$$0 \leq P \leq 1.000$$

Árvore Geradora Mínima (MST)

O problema é modelado como um grafo completo, onde cada casa é um vértice e cada par de casas tem uma aresta com custo igual à distância euclidiana entre elas:

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

As **K casas terrestres** e os **P cabos existentes** recebem custo zero — são tratados como conexões pré-estabelecidas no Union-Find antes de rodar o Kruskal.

A resposta é o peso total das arestas adicionadas pelo algoritmo de Kruskal para completar a MST.

- Inicializar Union-Find**
N nós, com compressão de caminho e união por rank
- Pré-conectar K casas terrestres**
`union(0, i)` para `i = 1..K-1` → custo zero
- Pré-conectar P cabos existentes**
`union(u-1, v-1)` para cada cabo → custo zero
- Gerar $N(N-1)/2$ arestas**
distância euclidiana entre todos os pares
- Ordenar e aplicar Kruskal**
somar distância quando `union()` retorna True

Casos de Teste

Exemplo 01

Sem conexões prévias

```
3 1 0
0,0 0,0
2.0 0.0
1.0 2.0
```

SAIDA: 4.236067

3 casas, apenas 1 terrestre e nenhum cabo. O algoritmo precisa construir a MST completa conectando as 3 casas — soma as 2 arestas mais curtas que formam a árvore.

Exemplo 02

Com cabo existente

```
3 1 1
0,0 0,0
0,5 2,0
2,5 2,0
1 2
```

SAIDA: 2.000000

Cabo entre casas 1 e 2 já existe — custo zero. Basta conectar a casa 3 ao componente. O Kruskal adiciona apenas a aresta de menor custo até a casa 3.

Exemplo 03

Duas casas terrestres

```
3 2 0
0,0 0,0
2.0 0.0
1.0 2.0
```

SAIDA: 2.236067

Casas 1 e 2 já conectadas por terra. Só falta incluir a casa 3. O algoritmo seleciona o menor cabo possível até ela, resultando em comprimento menor que o exemplo 1.

Código em Python 3

Union-Find

```
class UnionFind:
    def __init__(self, size):
        self.parent = list(range(size))
        self.rank = [0] * size

    def find(self, node):
        if self.parent[node] != node:
            self.parent[node] = \
                self.find(self.parent[node])
        return self.parent[node]

    def union(self, a, b):
        ra = self.find(a)
        rb = self.find(b)
        if ra == rb: return False
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        return True
```

Kruskal principal

```
def solve(n, k, cables, coords):
    uf = UnionFind(n)

    # K casas terrestres: custo zero
    for i in range(1, k):
        uf.union(0, i)

    # P cabos existentes: custo zero
    for u, v in cables:
        uf.union(u-1, v-1)

    # Gerar, ordenar e aplicar Kruskal
    edges = []
    for i in range(n):
        for j in range(i+1, n):
            d = dist(coords[i], coords[j])
            edges.append((d, i, j))
    edges.sort()

    total = 0.0
```

Desempenho e Resultados

Geração de arestas

$O(N^2)$

$N(N-1)/2$ pares calculados

Ordenação

$O(N^2 \log N)$

etapa dominante do algoritmo

Union-Find

$\approx O(N^2)$

quase linear por operação

Complexidade total: **$O(N^2 \log N)$** — viável para $N = 1.000$ (~500 mil arestas)

Modelagem

O problema é naturalmente uma MST. Cada casa vira um vértice; o custo de conexão é a distância euclidiana.

Insight-chave

Casas terrestres e cabos existentes são conexões de custo zero — simplificam o problema antes de qualquer cálculo.

Resultado

Solução aceita no Kattis com Kruskal + Union-Find em Python 3, dentro dos limites de tempo.